



Complex Computing Problem(CCP)

Name : Luqman Khan

SID : S20242660554

Course: Operating Systems

Complex Computing Problem Solution

1. Introduction

A university online course registration system receives thousands of student requests during enrollment. Each course has a limited number of seats. When many students try to register for the same course at the same time, multiple threads may access and update the same seat count.

Let's take an example where a course has just one seat remaining. Both pupils may notice that the seat is available if they check it simultaneously. Both students could be registered for the same final seat if there is no synchronization. As a result, a race situation is created.

Additionally, if the system generates a new thread for each student request, it might generate 5000 threads for 5000 student requests. This could overload CPU and memory and result in a crash.

So, solution must use:

- Mutex lock to prevent race conditions.
- Thread pool to avoid creating too many threads.
- Request queue to process requests fairly.
- FCFS scheduling to handle requests in arrival order.
- Fixed lock ordering to prevent deadlock during multiple-course registration.

2. Race Condition

A race condition occurs when two or more threads access shared data at the same time and the final result depends on the order of execution.

In this problem, the shared data is:

course seat count

Example:

Suppose course OS101 has only 1 seat left.

Student A and Student B both try to register at the same time.

Without synchronization:

Student A checks seats: seats = 1

Student B checks seats: seats = 1

Student A registers and decreases seats to 0

Student B also registers and decreases seats to -1

Now two students got one last seat. This is incorrect.

So, the system needs synchronization to ensure that only one student can check and update the course seat at one time.

3. Critical Section

Critical section is the part of the program where shared data is accessed or modified.

In this course registration system, the critical section is:

```
if (courseSeats[course] > 0) {  
    courseSeats[course]--;  
    registerStudent(studentId, course);  
}
```

This part is critical because multiple threads may try to:

- Check available seats.
- Decrease seat count.
- Register student.

If this section is not protected, race conditions can occur.

So, this critical section must be protected using a mutex lock.

4. Mutex Lock Solution

A **mutex lock** allows only one thread to enter the critical section at a time.

For each course, we use a separate mutex.

Example:

```
mutex courseMutex;
```

When a student wants to register for a course:

```
courseMutex.lock();
```

```
if (seats > 0) {
```

```
    seats--;
```

```
    cout << "Registration successful";
```

```
}
```

```
courseMutex.unlock();
```

Better C++ style is using:

```
lock_guard<mutex> lock(courseMutex);
```

This automatically locks and unlocks the mutex.

Purpose of Mutex Lock

The mutex lock is used to:

- Prevent two students from getting the same seat.
- Protect shared course seat count.
- Ensure safe update of course data.
- Avoid race conditions.

5. Thread Pool Solution

A **thread pool** is a fixed number of worker threads created in advance.

Instead of creating one thread for each student request, the system creates a limited number of worker threads, such as:

100 worker threads

If 5000 student requests arrive, they are placed into a queue. The 100 worker threads pick requests from the queue and process them one by one.

Why Thread Pool is Needed

Without a thread pool:

5000 requests = 5000 threads

This can cause:

- High CPU usage.
- High memory usage.
- Too much context switching.
- Server crash.

With a thread pool:

5000 requests + 100 worker threads

The system remains stable because only 100 threads are active.

Purpose of Thread Pool

The thread pool is used to:

- Prevent server crash.
- Control CPU and memory usage.
- Handle high traffic efficiently.
- Reuse existing threads.
- Improve system performance.

6. Request Queue

A **request queue** stores student registration requests.

The queue follows **FCFS**, which means:

First Come First Serve

The request that arrives first is processed first.

Example:

Request 1: Student 101 wants OS101

Request 2: Student 102 wants DB101

Request 3: Student 103 wants OS101

The system processes them in this order:

Student 101 → Student 102 → Student 103

Purpose of Request Queue

The request queue is used to:

- Store all incoming requests.
- Manage high traffic.
- Process requests fairly.
- Ensure FCFS scheduling.
- Avoid losing requests during peak time.

7. Deadlock Prevention

A **deadlock** can occur when two or more threads wait for each other forever.

Example:

Student A wants to register for:

OS101 and DB101

Student B wants to register for:

DB101 and OS101

Possible deadlock:

Student A locks OS101 and waits for DB101.

Student B locks DB101 and waits for OS101.

Both are waiting forever.

Deadlock Prevention using Fixed Lock Ordering

To prevent deadlock, all threads must lock courses in the same fixed order.

For example, sort course names alphabetically before locking:

DB101 first

OS101 second

So, every student request locks courses in the same order.

This removes circular waiting.

Purpose of Fixed Lock Ordering

Fixed lock ordering is used to:

- Prevent circular wait.
- Avoid deadlock.
- Safely register students in multiple courses.
- Ensure system stability.

8. OS Concepts Used

Problem	OS Concept Used
Two students getting same seat->	Mutex Lock / Synchronization
High traffic ->	Thread Pool
Too many requests ->	Request Queue
Fair registration ->	FCFS Scheduling
Multiple course registration deadlock ->	Deadlock Prevention using Fixed Lock Ordering

9. Performance Analysis of Course Registration System

Case	Worker Threads	Total Requests	Average Response Time	CPU Usage	Memory Usage
A	50	5000	9 seconds	55%	Low
B	100	5000	5 seconds	75%	Medium

C	500	5000	4 seconds	95%	High
D	5000	5000	System crash	100%	Very High

1. Which thread pool size is most suitable for this system and why?

The most suitable thread pool size is:

Case B = 100 worker threads

Case B is best because:

- Response time is good: **5 seconds**.
- CPU usage is acceptable: **75%**.
- Memory usage is medium.
- System does not crash.
- It gives a good balance between performance and resource usage.

Case C has slightly better response time, but CPU usage is very high. Case D crashes the system.

So, **100 worker threads** is the best choice.

2. Why does the system crash when 5000 threads are created?

The system crashes in Case D because creating **5000 threads** causes too much load.

Each thread needs:

- Stack memory.
- CPU scheduling.
- Context switching.

- Operating system resources.

When 5000 threads run at the same time:

- CPU usage becomes 100%.
- Memory usage becomes very high.
- Context switching overhead increases.
- The operating system cannot manage all threads efficiently.

So, the system crashes.

3. How does the mutex lock affect system performance?

A mutex lock improves correctness but can reduce speed slightly.

When one thread is inside the critical section, other threads must wait.

This waiting can increase response time.

However, mutex lock is necessary because it prevents:

- Race condition.
- Wrong seat count.
- Two students getting same seat.

So, mutex lock may reduce performance slightly, but it improves correctness and data safety.

4. What is the trade-off between response time and resource usage?

The trade-off is:

More threads = lower response time but higher CPU and memory usage

Fewer threads = lower CPU and memory usage but higher response time

Example:

- Case A uses 50 threads, so memory is low, but response time is 9 seconds.
- Case B uses 100 threads, so response time improves to 5 seconds.
- Case C uses 500 threads, so response time improves to 4 seconds, but CPU usage becomes 95%.
- Case D uses 5000 threads, so the system crashes.

Therefore, we need a balanced number of threads.

5. Why is Case B better than Case C, even though Case C has slightly better response time?

Case B is better because it gives a better balance.

Case C has response time:

4 seconds

Case B has response time:

5 seconds

The difference is only 1 second.

But Case C uses:

95% CPU

High memory

Case B uses:

75% CPU

Medium memory

So, Case B is more stable and efficient.

Case C puts too much load on the system and may cause performance problems if traffic increases.

6. Suggest two improvements to increase performance without causing system failure.

Improvement 1: Use a fixed-size thread pool

Use a controlled number of worker threads, such as:

100 worker threads

This prevents the system from creating too many threads and crashing.

Improvement 2: Use efficient locking

Use one mutex per course instead of one global mutex.

This allows students registering for different courses to be processed at the same time.

Example:

Student A registers for OS101.

Student B registers for DB101.

Both can proceed at the same time because they use different course locks.

Other possible improvements:

- Use database transactions.
- Use caching for course availability.
- Use load balancing.
- Use priority queue for special cases.
- Use asynchronous request handling.

10. Implementation in C++

CODE :

```
#include <iostream>
#include <thread>
#include <mutex>
#include <condition_variable>
#include <queue>
#include <vector>
#include <map>
#include <memory>
#include <algorithm>
#include <atomic>
using namespace std;

struct Course {
    string courseName;
    int seats;
    mutex courseMutex;

    Course(string name, int s) {
        courseName = name;
        seats = s;
    }
};

struct Request {
    int studentId;
    vector<string> courseNames;
};

map<string, shared_ptr<Course> > courses;

queue<Request> requestQueue;

mutex queueMutex;
condition_variable cv;
```

```
bool stopProcessing = false;
```

```
atomic<int> successfulRequests(0);
```

```
atomic<int> failedRequests(0);
```

```
atomic<int> totalProcessed(0);
```

```
void processRequest(Request req) {
```

```
    sort(req.courseNames.begin(), req.courseNames.end());
```

```
    vector<unique_lock<mutex> > locks;
```

```
    for (int i = 0; i < req.courseNames.size(); i++) {
```

```
        string cname = req.courseNames[i];
```

```
        if (courses.find(cname) != courses.end()) {
```

```
            locks.push_back(unique_lock<mutex>(courses[cname]->courseMutex));
```

```
        }
```

```
    }
```

```
    bool canRegister = true;
```

```
    for (int i = 0; i < req.courseNames.size(); i++) {
```

```
        string cname = req.courseNames[i];
```

```
        if (courses.find(cname) == courses.end()) {
```

```
            canRegister = false;
```

```
            break;
```

```
        }
```

```
        if (courses[cname]->seats <= 0) {
```

```
            canRegister = false;
```

```
            break;
```

```
        }
```

```
    }
```

```

if (canRegister) {
    for (int i = 0; i < req.courseNames.size(); i++) {
        string cname = req.courseNames[i];
        courses[cname]->seats--;
    }

    successfulRequests++;

    cout << "Student " << req.studentId << " registered successfully for: ";

    for (int i = 0; i < req.courseNames.size(); i++) {
        cout << req.courseNames[i] << " ";
    }

    cout << endl;
} else {
    failedRequests++;

    cout << "Student " << req.studentId << " registration failed for: ";

    for (int i = 0; i < req.courseNames.size(); i++) {
        cout << req.courseNames[i] << " ";
    }

    cout << endl;
}

totalProcessed++;
}

void workerThread() {
    while (true) {
        Request req;

        {
            unique_lock<mutex> lock(queueMutex);

            cv.wait(lock, [] {
                return !requestQueue.empty() || stopProcessing;
            });

```

```

        if (stopProcessing && requestQueue.empty()) {
            return;
        }

        req = requestQueue.front();
        requestQueue.pop();
    }

    processRequest(req);
}

}

void addRequest(Request req) {
    {
        lock_guard<mutex> lock(queueMutex);
        requestQueue.push(req);
    }

    cv.notify_one();
}

int main() {
    cout << "University Online Course Registration System\n";
    cout << "Using Mutex Locks, Thread Pool, Request Queue, FCFS, and Deadlock
Prevention\n\n";

    courses["CS101"] = make_shared<Course>("CS101", 100);
    courses["OS301"] = make_shared<Course>("OS301", 80);
    courses["DB201"] = make_shared<Course>("DB201", 60);
    courses["AI401"] = make_shared<Course>("AI401", 40);

    int totalRequests = 5000;
    int workerThreads = 100; // Case B: Best balanced option

    vector<thread> workers;

    for (int i = 0; i < workerThreads; i++) {
        workers.push_back(thread(workerThread));
    }
}

```



```

for (int i = 1; i <= totalRequests; i++) {
    Request req;
    req.studentId = i;

    if (i % 4 == 0) {
        req.courseNames.push_back("CS101");
    } else if (i % 4 == 1) {
        req.courseNames.push_back("OS301");
    } else if (i % 4 == 2) {
        req.courseNames.push_back("DB201");
    } else {
        req.courseNames.push_back("AI401");
    }

    if (i % 100 == 0) {
        req.courseNames.push_back("OS301");
        req.courseNames.push_back("DB201");
    }

    addRequest(req);
}

{
    unique_lock<mutex> lock(queueMutex);
    stopProcessing = true;
}

cv.notify_all();

for (int i = 0; i < workers.size(); i++) {
    workers[i].join();
}

cout << "\n===== Final Report =====\n";
cout << "Total Requests Processed: " << totalProcessed << endl;
cout << "Successful Registrations: " << successfulRequests << endl;
cout << "Failed Registrations: " << failedRequests << endl;

cout << "\nRemaining Course Seats:\n";

```

```

    for (auto it = courses.begin(); it != courses.end(); it++) {
        cout << it->first << " remaining seats: " << it->second->seats << endl;
    }

    cout << "\nSystem completed without race condition, deadlock, or thread
overload.\n";

    return 0;
}

```

Screenshot :

The screenshot displays a C++ IDE with two panels: 'main.cpp' on the left and 'Output' on the right.

main.cpp:

```

1  #include <iostream>
2  #include <thread>
3  #include <mutex>
4  #include <condition_variable>
5  #include <queue>
6  #include <vector>
7  #include <map>
8  #include <memory>
9  #include <algorithm>
10 #include <atomic>
11 using namespace std;
12
13 struct Course {
14     string courseName;
15     int seats;
16     mutex courseMutex;
17
18     Course(string name, int s) {
19         courseName = name;
20         seats = s;
21     }
22 };
23
24

```

Output:

```

University Online Course Registration System
Using Mutex Locks, Thread Pool, Request Queue, FCFS, and Deadlock Prevention

Student 1 registered successfully for: OS301
Student 3 registered successfully for: AI401
Student 2 registered successfully for: DB201
Student 5 registered successfully for: OS301
Student 4 registered successfully for: CS101
Student 7 registered successfully for: AI401
Student 8 registered successfully for: CS101
Student 9 registered successfully for: OS301
Student 6 registered successfully for: DB201
Student 10 registered successfully for: DB201
Student 11 registered successfully for: AI401
Student 12 registered successfully for: CS101
Student 13 registered successfully for: OS301
Student 14 registered successfully for: DB201
Student 15 registered successfully for: AI401
Student 17 registered successfully for: OS301
Student 16 registered successfully for: CS101
Student 18 registered successfully for: DB201
Student 19 registered successfully for: AI401
Student 20 registered successfully for: CS101
Student 21 registered successfully for: OS301

```

```
main.cpp  Run  Output
25- struct Request {
26     int studentId;
27     vector<string> courseNames;
28 };
29
30 map<string, shared_ptr<Course> > courses;
31
32 queue<Request> requestQueue;
33
34 mutex queueMutex;
35 condition_variable cv;
36
37 bool stopProcessing = false;
38
39
40 atomic<int> successfulRequests(0);
41 atomic<int> failedRequests(0);
42 atomic<int> totalProcessed(0);
43
44
45- void processRequest(Request req) {
46     sort(req.courseNames.begin(), req.courseNames.end());
47
48     Student 18 registered successfully for: DB201
49     Student 19 registered successfully for: AI401
50     Student 20 registered successfully for: CS101
51     Student 21 registered successfully for: OS301
52     Student 22 registered successfully for: DB201
53     Student 23 registered successfully for: AI401
54     Student 24 registered successfully for: CS101
55     Student 25 registered successfully for: OS301
56     Student 27 registered successfully for: AI401
57     Student 28 registered successfully for: CS101
58     Student 29 registered successfully for: OS301
59     Student 26 registered successfully for: DB201
60     Student 30 registered successfully for: DB201
61     Student 31 registered successfully for: AI401
62     Student 32 registered successfully for: CS101
63     Student 33 registered successfully for: OS301
64     Student 34 registered successfully for: DB201
65     Student 35 registered successfully for: AI401
66     Student 36 registered successfully for: CS101
67     Student 37 registered successfully for: OS301
68     Student 38 registered successfully for: DB201
69     Student 39 registered successfully for: AI401
70     Student 40 registered successfully for: CS101
71     Student 41 registered successfully for: OS301
72     Student 42 registered successfully for: DB201
```

```
main.cpp  Run  Output
45- void processRequest(Request req) {
46     sort(req.courseNames.begin(), req.courseNames.end());
47
48     vector<unique_lock<mutex> > locks;
49
50
51
52-     for (int i = 0; i < req.courseNames.size(); i++) {
53         string cname = req.courseNames[i];
54
55-         if (courses.find(cname) != courses.end()) {
56             locks.push_back(unique_lock<mutex>(courses[cname]->courseMutex));
57         }
58     }
59
60
61     bool canRegister = true;
62
63-     for (int i = 0; i < req.courseNames.size(); i++) {
64         string cname = req.courseNames[i];
65
66-         if (courses.find(cname) == courses.end()) {
67             canRegister = false;
68             break;
69         }
70
71-         if (courses[cname]->seats <= 0) {
72             canRegister = false;
73             break;
74         }
75     }
76
77-     if (canRegister) {
78-         for (int i = 0; i < req.courseNames.size(); i++) {
79             string cname = req.courseNames[i];
80             courses[cname]->seats--;
81         }
82
83         successfulRequests++;
84
85         cout << "Student " << req.studentId << " registered successfully for: ";
86
87-         for (int i = 0; i < req.courseNames.size(); i++) {
88             cout << req.courseNames[i] << " ";
89         }
90
91         Student 39 registered successfully for: AI401
92         Student 40 registered successfully for: CS101
93         Student 41 registered successfully for: OS301
94         Student 42 registered successfully for: DB201
95         Student 43 registered successfully for: AI401
96         Student 44 registered successfully for: CS101
97         Student 45 registered successfully for: OS301
98         Student 46 registered successfully for: DB201
99         Student 47 registered successfully for: AI401
100        Student 48 registered successfully for: CS101
101        Student 49 registered successfully for: OS301
102        Student 50 registered successfully for: DB201
103        Student 51 registered successfully for: AI401
104        Student 52 registered successfully for: CS101
105        Student 53 registered successfully for: OS301
106        Student 54 registered successfully for: DB201
107        Student 55 registered successfully for: AI401
108        Student 56 registered successfully for: CS101
109        Student 57 registered successfully for: OS301
110        Student 58 registered successfully for: DB201
111        Student 59 registered successfully for: AI401
112        Student 60 registered successfully for: CS101
113        Student 61 registered successfully for: OS301
114        Student 62 registered successfully for: DB201
```

```
66-     if (courses.find(cname) == courses.end()) {
67         canRegister = false;
68         break;
69     }
70
71-     if (courses[cname]->seats <= 0) {
72         canRegister = false;
73         break;
74     }
75
76
77-     if (canRegister) {
78-         for (int i = 0; i < req.courseNames.size(); i++) {
79             string cname = req.courseNames[i];
80             courses[cname]->seats--;
81         }
82
83         successfulRequests++;
84
85         cout << "Student " << req.studentId << " registered successfully for: ";
86
87-         for (int i = 0; i < req.courseNames.size(); i++) {
88             cout << req.courseNames[i] << " ";
89         }
90
91         Student 55 registered successfully for: AI401
92         Student 60 registered successfully for: CS101
93         Student 61 registered successfully for: OS301
94         Student 62 registered successfully for: DB201
95         Student 63 registered successfully for: AI401
96         Student 64 registered successfully for: CS101
97         Student 65 registered successfully for: OS301
98         Student 66 registered successfully for: DB201
99         Student 67 registered successfully for: AI401
100        Student 68 registered successfully for: CS101
101        Student 69 registered successfully for: OS301
102        Student 70 registered successfully for: DB201
103        Student 71 registered successfully for: AI401
104        Student 72 registered successfully for: CS101
105        Student 73 registered successfully for: OS301
106        Student 74 registered successfully for: DB201
107        Student 75 registered successfully for: AI401
108        Student 76 registered successfully for: CS101
109        Student 77 registered successfully for: OS301
110        Student 78 registered successfully for: DB201
111        Student 81 registered successfully for: OS301
112        Student 90 registered successfully for: DB201
113        Student 89 registered successfully for: OS301
114        Student 82 registered successfully for: DB201
115        Student 80 registered successfully for: CS101
```

```
main.cpp
90
91     cout << endl;
92 } else {
93     failedRequests++;
94
95     cout << "Student " << req.studentId << " registration failed for: ";
96
97     for (int i = 0; i < req.courseNames.size(); i++) {
98         cout << req.courseNames[i] << " ";
99     }
100
101     cout << endl;
102 }
103
104 totalProcessed++;
105 }
106
107 void workerThread() {
108     while (true) {
109         Request req;
110
111         {
112             unique_lock<mutex> lock(queueMutex);
113
```

```
Student 89 registered successfully for: OS301
Student 82 registered successfully for: DB201
Student 80 registered successfully for: CS101
Student 79 registered successfully for: AI401
Student 83 registered successfully for: AI401
Student 86 registered successfully for: DB201
Student 87 registered successfully for: AI401
Student 92 registered successfully for: CS101
Student 91 registered successfully for: AI401
Student 93 registered successfully for: OS301
Student 85 registered successfully for: OS301
Student Student 88 registered successfully for: CS101
Student 97 registered successfully for: OS301
Student 96 registered successfully for: CS101
94 registered successfully for: DB201
Student 95 registered successfully for: AI401
Student 98 registered successfully for: DB201
Student 99 registered successfully for: AI401
Student 100 registered successfully for: CS101 DB201 OS301
Student 84 registered successfully for: CS101
Student 101 registered successfully for: OS301
Student 102 registered successfully for: DB201
Student 103 registered successfully for: AI401
Student 104 registered successfully for: CS101
```

```
main.cpp
110
111 {
112     unique_lock<mutex> lock(queueMutex);
113
114     cv.wait(lock, [] {
115         return !requestQueue.empty() || stopProcessing;
116     });
117
118     if (stopProcessing && requestQueue.empty()) {
119         return;
120     }
121
122     req = requestQueue.front();
123     requestQueue.pop();
124 }
125
126 processRequest(req);
127 }
128 }
129
130 void addRequest(Request req) {
131     {
132         lock_guard<mutex> lock(queueMutex);
133         requestQueue.push(req);
134     }
135
136     cv.notify_one();
137 }
138
139 int main() {
140     cout << "University Online Course Registration System\n";
141     cout << "Using Mutex Locks, Thread Pool, Request Queue, FCFS, and Deadlock
        Prevention\n\n";
142
143     courses["CS101"] = make_shared<Course>("CS101", 100);
144     courses["OS301"] = make_shared<Course>("OS301", 80);
145     courses["DB201"] = make_shared<Course>("DB201", 60);
146     courses["AI401"] = make_shared<Course>("AI401", 40);
147
148     int totalRequests = 5000;
149     int workerThreads = 100; // Case B: Best balanced option
150
151     vector<thread> workers;
152
153     for (int i = 0; i < workerThreads; i++) {
154         workers.push_back(thread(workerThread));
155     }
156
```

```
Student 104 registered successfully for: CS101
Student 106 registered successfully for: DB201
Student Student 107 registered successfully for: AI401
Student 108 registered successfully for: CS101
Student 111 registered successfully for: AI401
Student 110 registered successfully for: DB201
Student 112 registered successfully for: CS101
105 registered successfully for: OS301
Student 113 registered successfully for: OS301
Student 114 registered successfully for: DB201
Student 115 registered successfully for: AI401
Student 116 registered successfully for: CS101
Student 109 registered successfully for: OS301
Student 119 registered successfully for: AI401
Student 120 registered successfully for: CS101
Student 118 registered successfully for: DB201
Student 121 registered successfully for: OS301
Student 124 registered successfully for: CS101
Student Student 126 registered successfully for: DB201
Student 123 registered successfully for: AI401
Student 125 registered successfully for: OS301
Student 127 registered successfully for: AI401
Student 173 registered successfully for: OS301
Student 174 registered successfully for: CS101
```

```
134     }
135
136     cv.notify_one();
137 }
138
139 int main() {
140     cout << "University Online Course Registration System\n";
141     cout << "Using Mutex Locks, Thread Pool, Request Queue, FCFS, and Deadlock
        Prevention\n\n";
142
143     courses["CS101"] = make_shared<Course>("CS101", 100);
144     courses["OS301"] = make_shared<Course>("OS301", 80);
145     courses["DB201"] = make_shared<Course>("DB201", 60);
146     courses["AI401"] = make_shared<Course>("AI401", 40);
147
148     int totalRequests = 5000;
149     int workerThreads = 100; // Case B: Best balanced option
150
151     vector<thread> workers;
152
153     for (int i = 0; i < workerThreads; i++) {
154         workers.push_back(thread(workerThread));
155     }
156
```

```
Student 125 registered successfully for: OS301
Student 127 registered successfully for: AI401
Student 173 registered successfully for: OS301
Student 174 registered successfully for: DB201
Student Student 177 registered successfully for: OS301
Student 117 registered successfully for: OS301
175 registered successfully for: AI401
Student 201 registered successfully for: OS301
Student 172Student 137 registered successfully for: OS301
Student 141 registered successfully for: OS301
    registered successfully for: CS101
Student 135 registered successfully for: AI401
Student 205 registered successfully for: OS301
Student 139 registered successfully for: AI401
Student 143 registered successfully for: AI401
Student 209 registered successfully for: OS301
Student 147 registered successfully for: AI401
Student 145 registered successfully for: OS301
182 registered successfully for: DB201
Student 136 registered successfully for: CS101
Student 220 registered successfully for: CS101
Student 228 registered successfully for: CS101
Student 151 registered successfully for: AI401
Student 155 registered successfully for: AI401
```

```
153+   for (int i = 0; i < workerThreads; i++) {
154       workers.push_back(thread(workerThread));
155   }
156
157   for (int i = 1; i <= totalRequests; i++) {
158       Request req;
159       req.studentId = i;
160
161       if (i % 4 == 0) {
162           req.courseNames.push_back("CS101");
163       } else if (i % 4 == 1) {
164           req.courseNames.push_back("OS301");
165       } else if (i % 4 == 2) {
166           req.courseNames.push_back("DB201");
167       } else {
168           req.courseNames.push_back("AI401");
169       }
170
171       if (i % 100 == 0) {
172           req.courseNames.push_back("OS301");
173           req.courseNames.push_back("DB201");
174       }
175
176       addRequest(req);
177   }
```

Student 220 registered successfully for: CS101
Student 228 registered successfully for: CS101
Student 151 registered successfully for: AI401
Student 155 registered successfully for: AI401
Student 149 registered successfully for: OS301
Student 159 registered successfully for: AI401
Student 130 registered successfully for: DB201
Student 153 registered successfully for: OS301
Student Student 134 registered successfully for: DB201
Student 163 registration failed for: AI401
Student 157 registered successfully for: OS301
Student 138 registered successfully for: DB201
Student 161 registered successfully for: OS301
Student 142 registered successfully for: DB201
148 registered successfully for: CS101
Student 165 registered successfully for: OS301
Student 146 registered successfully for: DB201
Student 169 registered successfully for: OS301
Student 167 registration failed for: AI401
Student Student Student 171 registration failed for: AI401
152Student 179 registration failed for: AI401
Student Student 183 registration failed for: AI401
181 registered successfully for: OS301
Student 131 registration failed for: AI401
150 registered successfully for: DB201

```
main.cpp
170
171+   if (i % 100 == 0) {
172       req.courseNames.push_back("OS301");
173       req.courseNames.push_back("DB201");
174   }
175
176   addRequest(req);
177 }
178
179+ {
180     unique_lock<mutex> lock(queueMutex);
181     stopProcessing = true;
182 }
183
184 cv.notify_all();
185
186+ for (int i = 0; i < workers.size(); i++) {
187     workers[i].join();
188 }
189
190 cout << "\n===== Final Report =====\n";
191 cout << "Total Requests Processed: " << totalProcessed << endl;
192 cout << "Successful Registrations: " << successfulRequests << endl;
193 cout << "Failed Registrations: " << failedRequests << endl;
194 }
```

Student Student 183 registration failed for: AI401
181 registered successfully for: OS301
Student 131 registration failed for: AI401
150 registered successfully for: DB201
Student 238 registered successfully for: DB201
Student 235 registration failed for: AI401 Student
158 registered successfully for: DB201
Student 246 registered successfully for: DB201
Student 129 registered successfully for: OS301
registered successfully for: CS101
Student 156 registered successfully for: CS101
Student 166 registered successfully for: DB201
Student 170 registered successfully for: DB201
Student Student 275 registration failed for: AI401
160 registered successfully for: CS101
Student 195 registration failed for: AI401
Student Student 185 registered successfully for: OS301
Student 164199 registration failed for: AI401
registered successfully for: CS101
Student 189 registered successfully for: OS301
Student 283 registration failed for: AI401
Student 284 registered successfully for: CS101
Student 281 registered successfully for: OS301
Student 168 registered successfully for: CS101

```
main.cpp
182 }
183
184 cv.notify_all();
185
186+ for (int i = 0; i < workers.size(); i++) {
187     workers[i].join();
188 }
189
190 cout << "\n===== Final Report =====\n";
191 cout << "Total Requests Processed: " << totalProcessed << endl;
192 cout << "Successful Registrations: " << successfulRequests << endl;
193 cout << "Failed Registrations: " << failedRequests << endl;
194
195 cout << "\nRemaining Course Seats:\n";
196
197+ for (auto it = courses.begin(); it != courses.end(); it++) {
198     cout << it->first << " remaining seats: " << it->second->seats << endl;
199 }
200
201 cout << "\nSystem completed without race condition, deadlock, or thread
    overload.\n";
202
203 return 0;
204 }
```

Student 287 registration failed for: AI401
Student 197 registered successfully for: OS301
Student 288 registered successfully for: CS101
Student 133 registered successfully for: OS301
Student 128 registered successfully for: CS101
Student 203 registration failed for: AI401
Student 213 registered successfully for: OS301
Student 207 registration failed for: AI401
Student 292 registered successfully for: CS101
Student 184 registered successfully for: CS101
Student Student 188 registered successfully for: CS101
Student 274 registered successfully for: DB201
Student 298 registered successfully for: DB201
Student Student 217 registered successfully for: 186 registered successfully for:
OS301DB201
295 registration failed for: AI401
Student 301 registered successfully for: OS301
Student 303 registration failed for: AI401
Student 305 registered successfully for: OS301
Student 190 registered successfully for: DB201
Student 194 registered successfully for: DB201
Student 225 registered successfully for: OS301
Student Student 192 registered successfully for: 198 registered successfully for:

```

182     }
183
184     cv.notify_all();
185
186     for (int i = 0; i < workers.size(); i++) {
187         workers[i].join();
188     }
189
190     cout << "\n===== Final Report =====\n";
191     cout << "Total Requests Processed: " << totalProcessed << endl;
192     cout << "Successful Registrations: " << successfulRequests << endl;
193     cout << "Failed Registrations: " << failedRequests << endl;
194
195     cout << "\nRemaining Course Seats:\n";
196
197     for (auto it = courses.begin(); it != courses.end(); it++) {
198         cout << it->first << " remaining seats: " << it->second->seats << endl;
199     }
200
201     cout << "\nSystem completed without race condition, deadlock, or thread
        overload.\n";
202
203     return 0;
204 }

```

Student 30/ registration failed for: AI401
 Student 202 registered successfully for: DB201
 Student 196 registered successfully for: CS101
 Student 206 registered successfully for: DB201
 Student Student 233 registered successfully for: OS301
 Student 210 registered successfully for: DB201
 315 registration failed for: AI401 Student
 Student 237 registered successfully for: OS301 Student 223 registration failed for: AI401
 Student 316 registered successfully for: CS101
 214 registered successfully for: DB201
 Student 241 registered successfully for: OS301
 Student 245 registered successfully for: OS301
 Student 249 registered successfully for: OS301
 Student 325 registered successfully for: OS301
 Student 253 registered successfully for: OS301
 Student 329 registered successfully for: OS301
 Student 257 registered successfully for: OS301
 Student 327 registration failed for: AI401
 Student 261 registered successfully for: OS301
 Student 333 registered successfully for: OS301
 Student 269 registered successfully for: OS301
 Student 187 registration failed for: AI401

```

182     }
183
184     cv.notify_all();
185
186     for (int i = 0; i < workers.size(); i++) {
187         workers[i].join();
188     }
189
190     cout << "\n===== Final Report =====\n";
191     cout << "Total Requests Processed: " << totalProcessed << endl;
192     cout << "Successful Registrations: " << successfulRequests << endl;
193     cout << "Failed Registrations: " << failedRequests << endl;
194
195     cout << "\nRemaining Course Seats:\n";
196
197     for (auto it = courses.begin(); it != courses.end(); it++) {
198         cout << it->first << " remaining seats: " << it->second->seats << endl;
199     }
200
201     cout << "\nSystem completed without race condition, deadlock, or thread
        overload.\n";
202
203     return 0;
204 }

```

Student 339 registration failed for: AI401
 Student 337 registered successfully for: OS301
 Student 341 registered successfully for: OS301
 Student 277 registered successfully for: OS301
 Student 343 registration failed for: AI401
 Student 193 registered successfully for: OS301
 Student 239 registration failed for: AI401
 Student 345 registered successfully for: OS301
 Student 243 registration failed for: AI401
 Student 289 registered successfully for: OS301
 Student 247 registration failed for: AI401
 Student 349 registered successfully for: OS301
 Student 351 registration failed for: AI401
 Student 297 registration failed for: OS301
 Student 353 registration failed for: OS301
 Student 251 registration failed for: AI401
 Student 355 registration failed for: AI401
 Student 255 registration failed for: AI401
 Student 309 registration failed for: OS301
 Student 357 registration failed for: OS301
 Student 313 registration failed for: OS301
 Student 259 registration failed for: AI401
 Student 317 registration failed for: OS301
 Student 263 registration failed for: AI401

```

main.cpp
182     }
183
184     cv.notify_all();
185
186     for (int i = 0; i < workers.size(); i++) {
187         workers[i].join();
188     }
189
190     cout << "\n===== Final Report =====\n";
191     cout << "Total Requests Processed: " << totalProcessed << endl;
192     cout << "Successful Registrations: " << successfulRequests << endl;
193     cout << "Failed Registrations: " << failedRequests << endl;
194
195     cout << "\nRemaining Course Seats:\n";
196
197     for (auto it = courses.begin(); it != courses.end(); it++) {
198         cout << it->first << " remaining seats: " << it->second->seats << endl;
199     }
200
201     cout << "\nSystem completed without race condition, deadlock, or thread
        overload.\n";
202
203     return 0;
204 }

```

Student 363 registration failed for: AI401
 Student 200 registration failed for: CS101 DB201 OS301
 Student 267 registration failed for: AI401
 Student 366 registered successfully for: DB201
 Student Student 226 registered successfully for: DB201
 Student 365 registration failed for: OS301
 204 registered successfully for: CS101
 Student 265 registration failed for: OS301
 Student 271 registration failed for: AI401
 Student Student 208 registered successfully for: CS101
 Student 372 registered successfully for: CS101
 Student Student 216 registered successfully for: CS101
 369230 registration failed for: Student Student 3751400S301 registered successfully for:
 CS101
 Student 376 registered successfully for:
 CS101 Student 293 registration failed for:
 registration failed for: AI401
 Student 144 registered successfully for: CS101
 OS301
 Student Student 221 registration failed for: OS301
 Student 381 registration failed for: OS301
 279 registration failed for: AI401
 Student 383 registration failed for: AI401
 Student 321 registration failed for: OS301

```
main.cpp
182     }
183
184     cv.notify_all();
185
186     for (int i = 0; i < workers.size(); i++) {
187         workers[i].join();
188     }
189
190     cout << "\n===== Final Report =====\n";
191     cout << "Total Requests Processed: " << totalProcessed << endl;
192     cout << "Successful Registrations: " << successfulRequests << endl;
193     cout << "Failed Registrations: " << failedRequests << endl;
194
195     cout << "\nRemaining Course Seats:\n";
196
197     for (auto it = courses.begin(); it != courses.end(); it++) {
198         cout << it->first << " remaining seats: " << it->second->seats << endl;
199     }
200
201     cout << "\nSystem completed without race condition, deadlock, or thread
        overload.\n";
202
203     return 0;
204 }
```

Output

Student 230 registered successfully for: CS101
Student 285 registration failed for: OS301
Student 373 registration failed for: OS301
234Student 377 registration failed for: OS301
Student 361 registration failed for: OS301
registered successfully for: DB201
291 registration failed for: AI401Student 154 registered successfully for: DB201

Student 240 registered successfully for: CS101
Student 211 registration failed for: AI401
Student 242 registration failed for: DB201
Student Student 389 registration failed for: OS301
Student 244 registered successfully for: CS101
399Student registration failed for: AI401
Student 403 registration failed for: AI401
Student Student 311 registration failed for: AI401 Student
248 registered successfully for: CS101
393162 registration failed for: DB201
Student 406 registration failed for: DB201
Student registration failed for: OS301
Student 252 registered successfully for: CS101
Student 407 registration failed for: AI401
254 registration failed for: DB201
Student Student 409256 registered successfully for: CS101
Student 413 registered successfully for: CS101

```
main.cpp
182     }
183
184     cv.notify_all();
185
186     for (int i = 0; i < workers.size(); i++) {
187         workers[i].join();
188     }
189
190     cout << "\n===== Final Report =====\n";
191     cout << "Total Requests Processed: " << totalProcessed << endl;
192     cout << "Successful Registrations: " << successfulRequests << endl;
193     cout << "Failed Registrations: " << failedRequests << endl;
194
195     cout << "\nRemaining Course Seats:\n";
196
197     for (auto it = courses.begin(); it != courses.end(); it++) {
198         cout << it->first << " remaining seats: " << it->second->seats << endl;
199     }
200
201     cout << "\nSystem completed without race condition, deadlock, or thread
        overload.\n";
202
203     return 0;
204 }
```

Output

260 registered successfully for: CS101
OS301
Student 415 registration failed for: AI401
Student 268 registered successfully for: CS101
Student 418 registration failed for: DB201
Student 401 registration failed for: OS301
Student 266 registration failed for: DB201
Student 227 registration failed for: AI401
Student 270 registration failed for: DB201
Student 425 registration failed for: OS301
Student 426 registration failed for: DB201
Student 405 registration failed for: OS301
Student Student 429 registration failed for: OS301
427 registration failed for: AI401
Student 431 registration failed for: AI401
Student Student 447 registration failed for: AI401
Student 428 registered successfully for: CS101
Student 413 registration failed for: OS301
278 registration failed for: DB201
Student 433 registration failed for: OS301
Student 434 registration failed for: DB201
Student 436 registered successfully for: CS101
Student 438 registration failed for: DB201
Student 282 registration failed for: DB201

```
main.cpp
182     }
183
184     cv.notify_all();
185
186     for (int i = 0; i < workers.size(); i++) {
187         workers[i].join();
188     }
189
190     cout << "\n===== Final Report =====\n";
191     cout << "Total Requests Processed: " << totalProcessed << endl;
192     cout << "Successful Registrations: " << successfulRequests << endl;
193     cout << "Failed Registrations: " << failedRequests << endl;
194
195     cout << "\nRemaining Course Seats:\n";
196
197     for (auto it = courses.begin(); it != courses.end(); it++) {
198         cout << it->first << " remaining seats: " << it->second->seats << endl;
199     }
200
201     cout << "\nSystem completed without race condition, deadlock, or thread
        overload.\n";
202
203     return 0;
204 }
```

Output

Student 290 registration failed for: DB201
Student 440 registered successfully for: CS101
Student 294 registration failed for: DB201
Student 417443 registration failed for: registration failed for: OS301AI401

Student 176 registered successfully for: CS101
Student Student 421 registration failed for: OS301
Student 450 registration failed for: DB201
447Student 397 registration failed for: OS301
registration failed for: AI401
Student Student 441 registration failed for: OS301
Student 191 registration failed for: AI401
Student Student 276379 registration failed for: AI401
Student 387 registration failed for: AI401
306 registration failed for: DB201
Student Student 459 registration failed for: AI401
Student 310 registration failed for: DB201
registered successfully for: CS101
Student Student 180 registered successfully for: CS101
Student 296 registered successfully for: CS101
445 registration failed for: OS301
Student 465 registration failed for: OS301
Student 464Student 453 registration failed for: OS301
registered successfully for: Student 457 registration failed for: OS301


```

182     }
183
184     cv.notify_all();
185
186     for (int i = 0; i < workers.size(); i++) {
187         workers[i].join();
188     }
189
190     cout << "\n===== Final Report =====\n";
191     cout << "Total Requests Processed: " << totalProcessed << endl;
192     cout << "Successful Registrations: " << successfulRequests << endl;
193     cout << "Failed Registrations: " << failedRequests << endl;
194
195     cout << "\nRemaining Course Seats:\n";
196
197     for (auto it = courses.begin(); it != courses.end(); it++) {
198         cout << it->first << " remaining seats: " << it->second->seats << endl;
199     }
200
201     cout << "\nSystem completed without race condition, deadlock, or thread
        overload.\n";
202
203     return 0;
204 }

```

442 registration failed for: OS301
 Student 465 registration failed for: OS301
 Student 464Student 453 registration failed for: OS301
 registered successfully for: Student 457 registration failed for: OS301
 Student 302 registration failed for: DB201
 Student 461 registration failed for: OS301
 395 registration failed for: AI401
 CS101
 Student 471 registration failed for: AI401
 Student Student 472 registered successfully for: CS101 469 registration failed for
 Student 215 registration failed for: AI401
 OS301
 Student 314 registration failed for: DB201
 Student Student 476 registered successfully for: CS101 318
 registration failed for: Student 308 registered successfully for: CS101
 Student 449 registration failed for: OS301
 Student 132 registered successfully for: CS101
 DB201
 Student 219 registration failed for: AI401
 Student 473 registration failed for: OS301
 Student 482 registration failed for: DB201
 Student Student 218 registration failed for: DB201
 Student Student 486 registration failed for: DB201
 Student 485 registration failed for: OS301
 Student 488 registration failed for: OS301 registered successfully for: CS101

main.cpp

```

182     }
183
184     cv.notify_all();
185
186     for (int i = 0; i < workers.size(); i++) {
187         workers[i].join();
188     }
189
190     cout << "\n===== Final Report =====\n";
191     cout << "Total Requests Processed: " << totalProcessed << endl;
192     cout << "Successful Registrations: " << successfulRequests << endl;
193     cout << "Failed Registrations: " << failedRequests << endl;
194
195     cout << "\nRemaining Course Seats:\n";
196
197     for (auto it = courses.begin(); it != courses.end(); it++) {
198         cout << it->first << " remaining seats: " << it->second->seats << endl;
199     }
200
201     cout << "\nSystem completed without race condition, deadlock, or thread
        overload.\n";
202
203     return 0;
204 }

```

Student Student 218 registration failed for: DB201
 Student Student 486 registration failed for: DB201
 Student 485 registration failed for: OS301
 480Student 322 registration failed for: DB201 registered successfully for: CS101
 Student 481 registration failed for: OS301
 Student 489 registration failed for: OS301
 319Student registration failed for: AI401
 Student 492 registered successfully for: CS101
 Student 419 registration failed for: AI401
 477 registration failed for: OS301
 Student 493 registration failed for: OS301
 Student 326 registration failed for: DB201
 Student 497 registration failed for: OS301
 Student 498 registration failed for: DB201
 Student 499 registration failed for: AI401
 Student 496 registered successfully for: CS101
 Student 323 registration failed for: AI401
 Student 502 registration failed for: DB201
 Student Student 501 registration failed for: OS301
 Student 334 registration failed for: DB201
 Student 503 registration failed for: AI401
 Student 507 registration failed for: AI401
 Student 506 registration failed for: DB201

```

182     }
183
184     cv.notify_all();
185
186     for (int i = 0; i < workers.size(); i++) {
187         workers[i].join();
188     }
189
190     cout << "\n===== Final Report =====\n";
191     cout << "Total Requests Processed: " << totalProcessed << endl;
192     cout << "Successful Registrations: " << successfulRequests << endl;
193     cout << "Failed Registrations: " << failedRequests << endl;
194
195     cout << "\nRemaining Course Seats:\n";
196
197     for (auto it = courses.begin(); it != courses.end(); it++) {
198         cout << it->first << " remaining seats: " << it->second->seats << endl;
199     }
200
201     cout << "\nSystem completed without race condition, deadlock, or thread
        overload.\n";
202
203     return 0;
204 }

```

Student 2178 registration failed for: DB201
 Student 2179 registration failed for: AI401
 Student 2180 registration failed for: CS101
 Student 2181 registration failed for: OS301
 Student 2182 registration failed for: DB201
 Student 2183 registration failed for: AI401
 Student 2184 registration failed for: CS101
 Student 2185 registration failed for: OS301
 Student 2186 registration failed for: DB201
 Student 2187 registration failed for: AI401
 Student 2188 registration failed for: CS101
 Student 2189 registration failed for: OS301
 Student 2190 registration failed for: DB201
 Student 2191 registration failed for: AI401
 Student 2192 registration failed for: CS101
 Student 2193 registration failed for: OS301
 Student 2194 registration failed for: DB201
 Student 2195 registration failed for: AI401
 Student 2196 registration failed for: CS101
 Student 2197 registration failed for: OS301
 Student 2198 registration failed for: DB201

=== Execution Halted ===